

Data-driven Invariant Learning for Probabilistic Programs

Jialu Bao

PLDG, March 9th, 2022

Collaborated work with Nitesh Trivedi, Drashti Pathak, Justin Hsu, Subhajit Roy

Motivation

Ex. 1 $\mathbb{E}(z) = z + p$

$z \leftarrow z + 1$ [p] skip

What is the expected value of z at
the end of program in term of
program variables' value at the
initialization?

Ex. 2 $\mathbb{E}(z) = z + [\text{flip} == 0] \cdot (1 - p)/p$

while (flip == 0) do

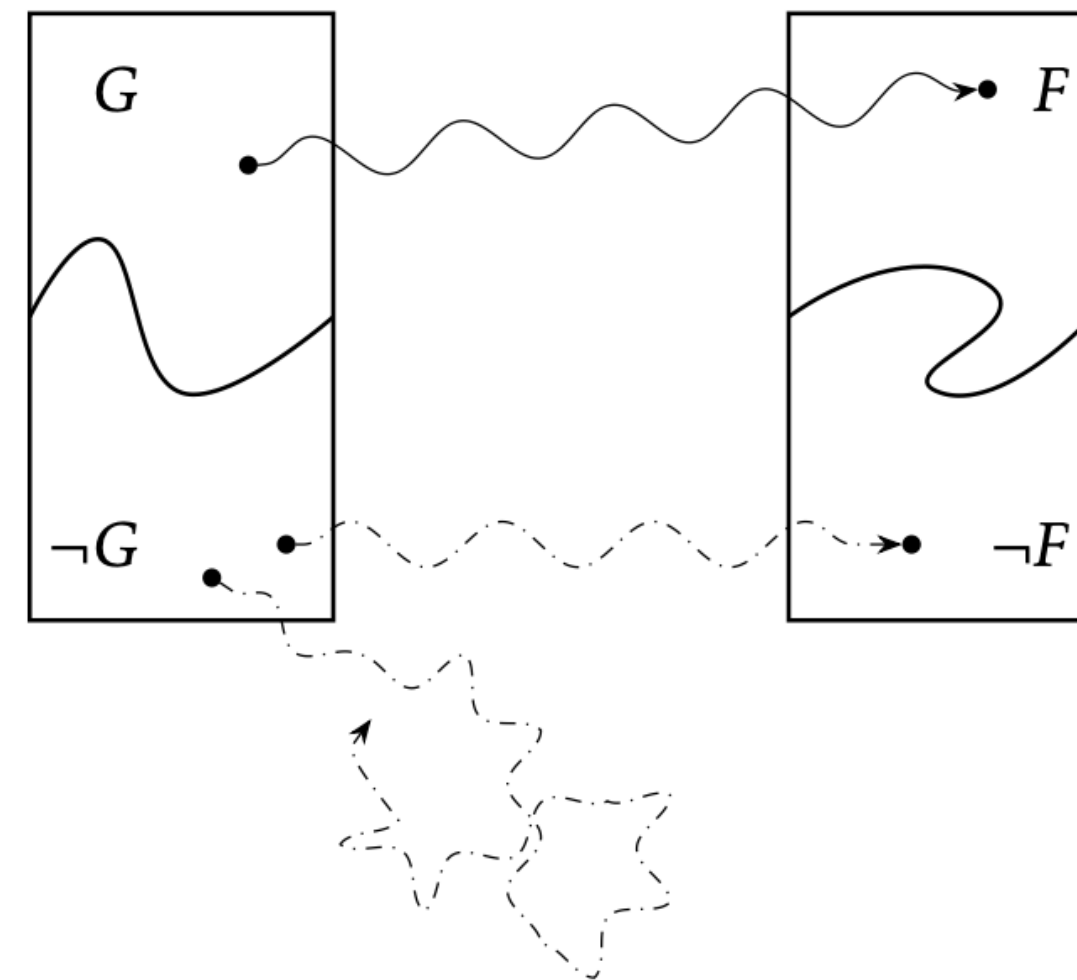
flip $\leftarrow$ 1 [p] $z \leftarrow z + 1$

Can we automatically find
this answer?

Background: **W**eakest **P**re-**c**ondition Calculus

- Edsger Dijkstra, 1975
- $wpc : \text{Programs} \times \text{Assertions} \rightarrow \text{Assertions}$

- Say $wpc(C, F) = G$



- Example rules:

- Assignment: $wpc(x \leftarrow a, F) := F[a/x]$

- Sequencing: $wpc(P; Q, F) := wpc(P, wpc(Q, F))$

Background: Weakest Pre-expectation Calculus

- Kozen, ~1985; McIver & Morgan and others, 1990s~today
- $wpe : \text{Programs} \times \text{Expectations} \rightarrow \text{Expectations}$
- Expectations are maps from program states to numbers
 - Ex. $st \mapsto 2 \cdot st[x]$, or $st \mapsto st[x] + st[y]$
 - We simply write $2 \cdot x$, or $x + y$
- Iverson bracket: $[G]$ maps states where the assertion G holds to 1 and maps other states to 0

Reasoning about Quantities

- We let $wpe(C, e)$ be the expected value of expectation e after running C
 1. It generalizes weakest pre-condition calculus, e.g.,
 - $wpc(x \leftarrow x + 1, x = y)$ is $x + 1 = y$
 - $wpe(x \leftarrow x + 1, [x = y])$ is $[x + 1 = y]$
 2. Expected values are useful.
 3. It can encode probability of an event Ev in the output distribution:
 - Let the expectation e be $[Ev]$.

Weakest Pre-expectation Calculus

reason about expected values!

Definition [Morgan and McIver]

$$wpe(x \leftarrow a, e) := e[a/x]$$

$$wpe(P; Q, e) := wpe(P, wpe(Q, e))$$

$$wpe(C[p]C', e) := p \cdot wpe(C, e) + (1 - p) \cdot wpe(C', e)$$

$$wpe(\mathbf{while} \ b \ \mathbf{do} \ body, e) := \mathbf{lfp}(\lambda x. [b] \cdot wpe(body, x) + [\neg b] \cdot e)$$

...

Theorem. $wpe(C, e) = \lambda s. \text{ expected value of } e \text{ after running } C \text{ from } s$

WPE Calculus on Motivating Examples

Ex. 1

$$wpe(z \leftarrow z + 1 \text{ [p] skip}, z)$$

$$= p \cdot wpe(z \leftarrow z + 1, z) + (1 - p) \cdot wpe(\text{skip}, z)$$

$$= p \cdot (z + 1) + (1 - p) \cdot z$$

$$= z + p$$

The least fixed point is indirect and hard to work with!

Ex. 2

$$wpe(\text{while } b \text{ do } body, e) := \text{lfp}(\lambda x. [b] \cdot wpe(body, x) + [\neg b] \cdot e)$$

$$wpe(\text{while } (\text{flip} == 0) \text{ do } (\text{flip} \leftarrow 1 \text{ [p] } z \leftarrow z + 1), z)?$$

ϕ

Problem Statement

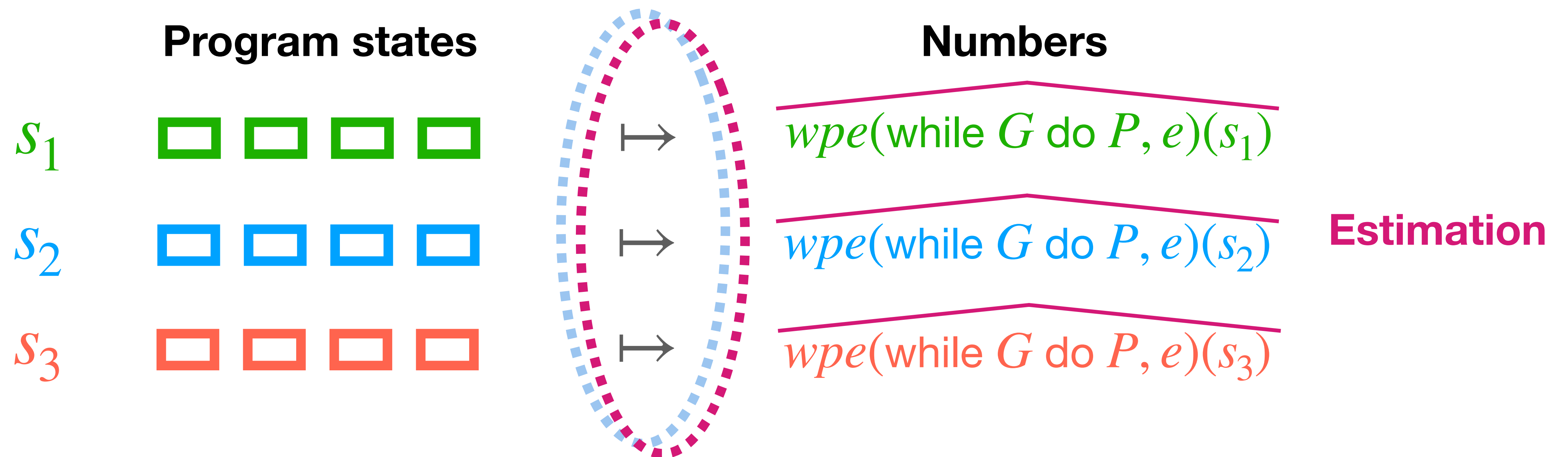
- Given: a loop **while** G **do** P and an expectation e .
- Goal: find expectation I such that $I = [G] \cdot wpe(P, I) + [\neg G] \cdot e$.
- We call e the postexpectation, and call I an **exact invariant** of the loop.

Why find exact invariant?

- For simple cases, there is unique fixed point for ϕ , so I is a fixed point of ϕ iff I is the least fixed point.
- In other words, in simple cases, I is an exact invariant iff $I = wpe(\text{while } G \text{ do } P, e)$.
- Simple: the loop is almost surely terminating and e is upper bounded by a constant

Casting into a Learning Problem

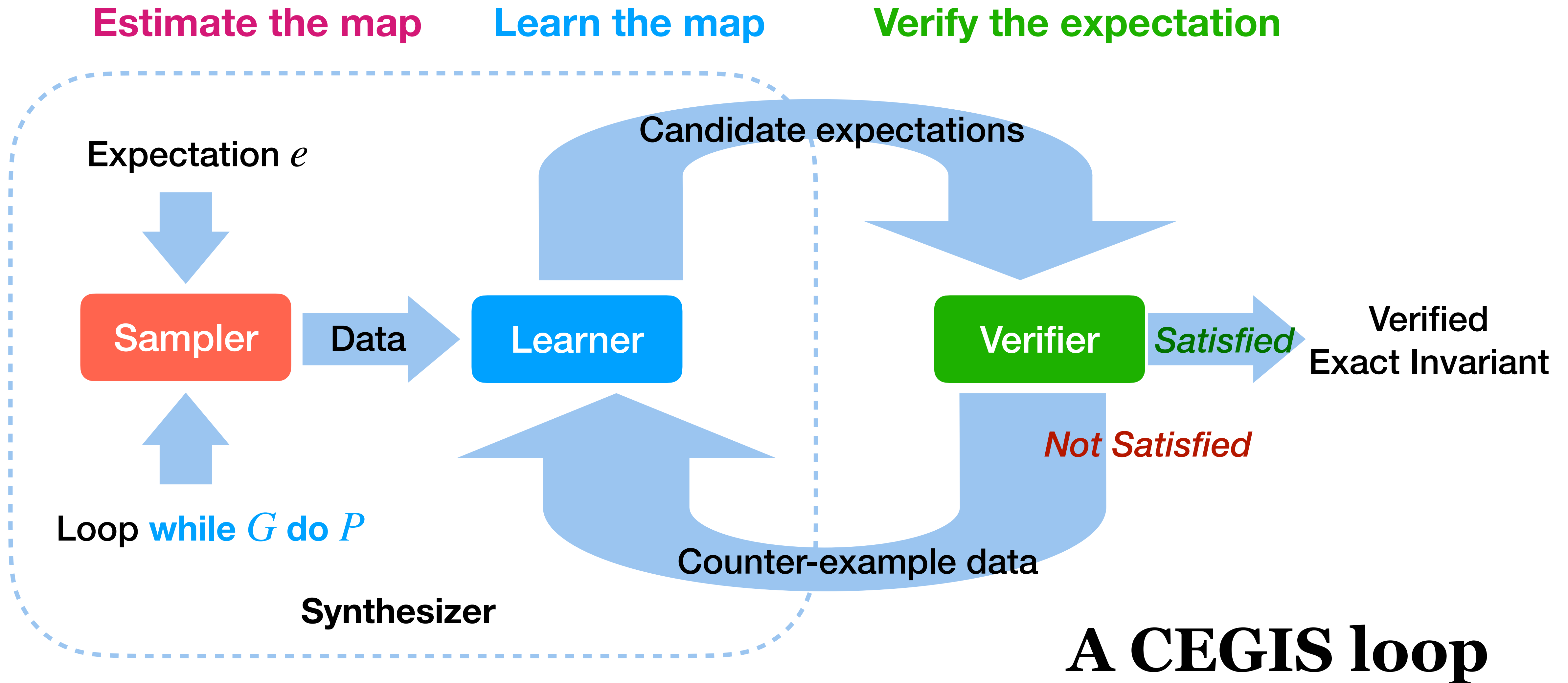
- $I = wpe(\text{while } G \text{ do } P, e)$ is a map from program states to numbers



If we learn this map, it approximates $wpe(\text{while } G \text{ do } P, e)$ on sampled program states

The learned map may not be $wpe(\text{while } G \text{ do } P, e)$ but we can check whether it is an exact invariant.

Method Overview



Sampler

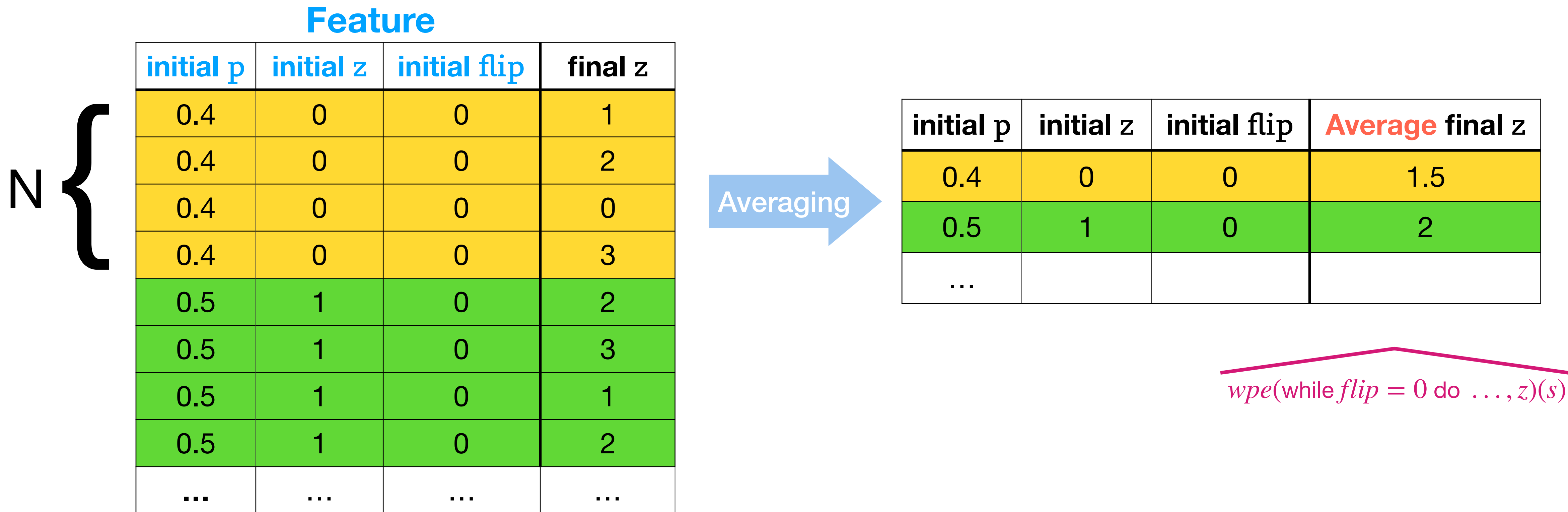
- How to estimate $wpe(\text{while } G \text{ do } P, e)(s)$?
 - It is the **expected value** of e on the distribution obtained from running while G do P from s .
 - We can approximate expected values by **empirical means**.

Sampler: Sample Program States

- To sample a program state, we sample each program variable's value from a range:
 - bool: uniformly from $\{0,1\}$
 - int: uniformly from $\{0,1,\dots,20\}$
 - prob: uniformly from $[0.01,0.99]$
 - ...
- Sample M program states in total

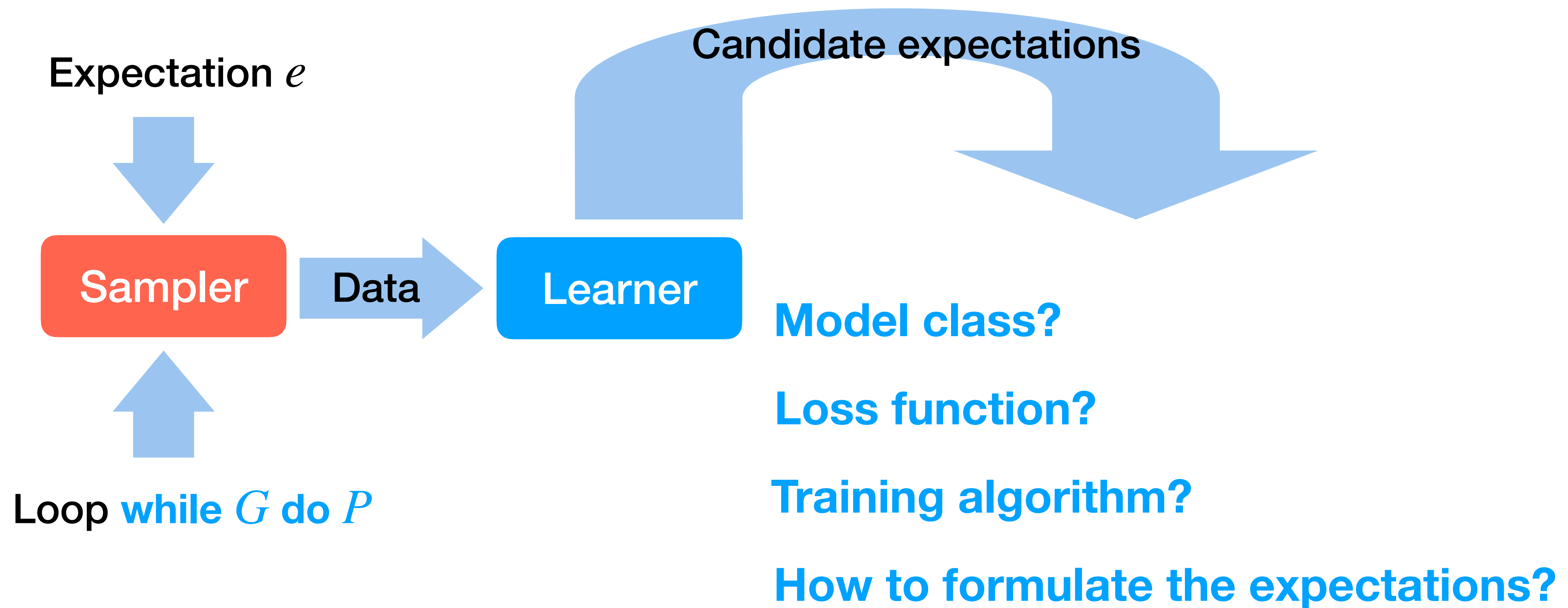
Sampler: Estimate wpe on program states

- Generate a list of features
- For each sampled program state s , run the **loop** for N times and record the postexpectation's value when the loop exits



Estimate the map

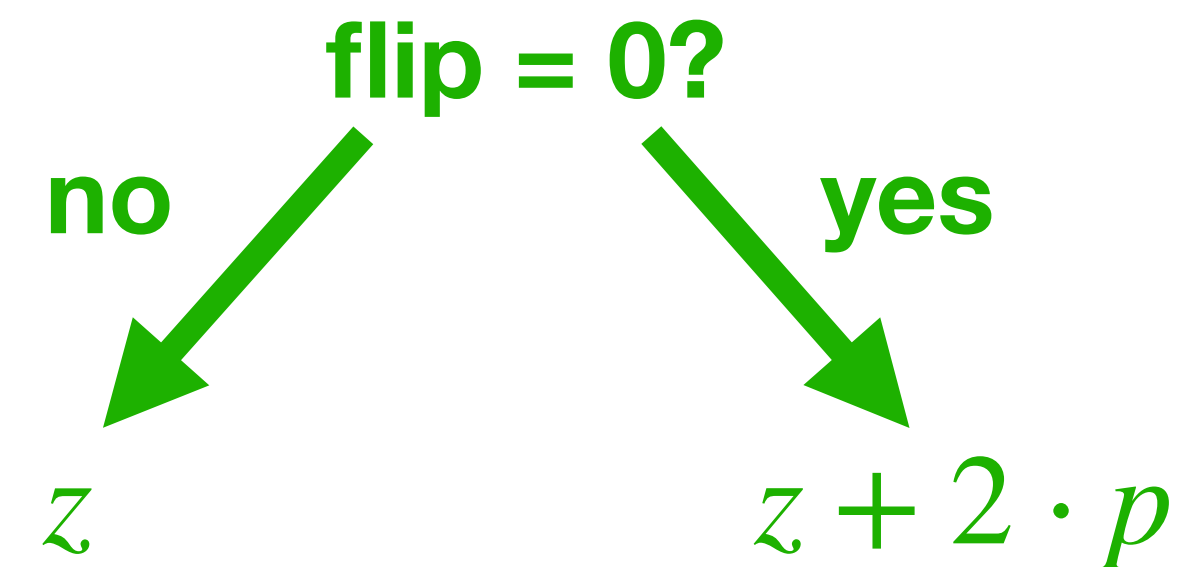
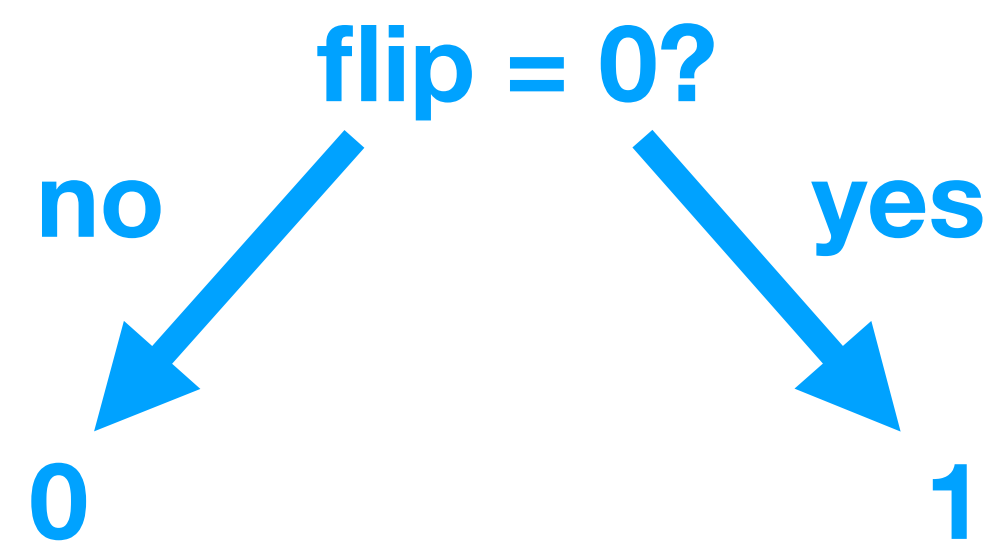
Learn the map



Learner: Model Class

- **Model trees** generalize **decision trees** with a model at each leaf.

- Ex.



- Leaf models: linear functions or cumulative products of features
- Advantages:
 - Easy for human to interpret
 - Easy for verifier to manipulate

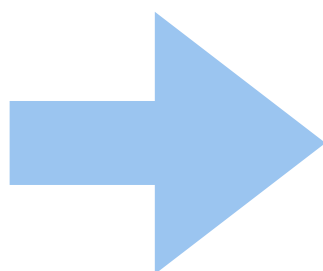
Learner: Loss and Training

- Loss of a model tree T on collected data D

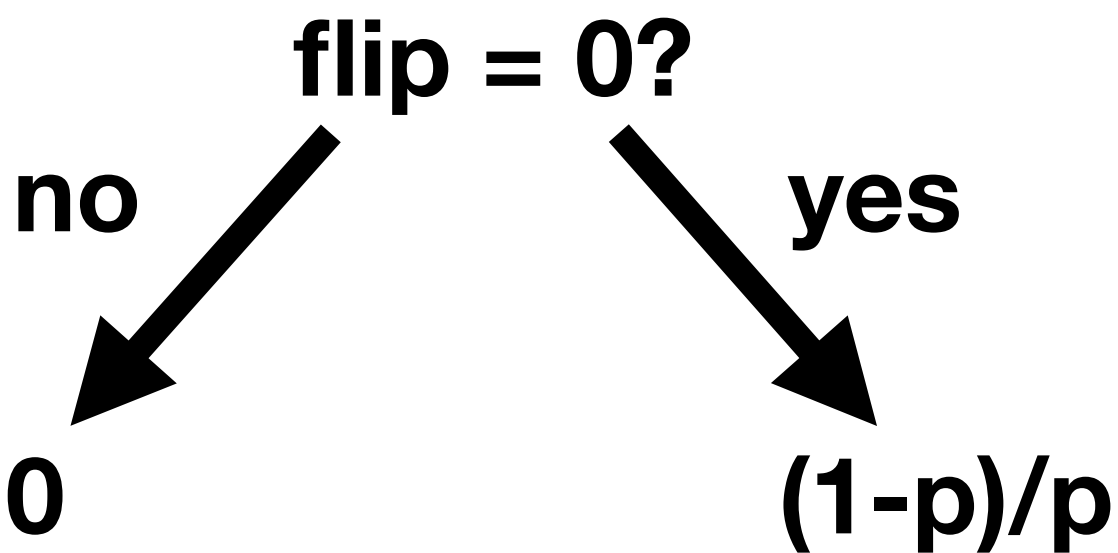
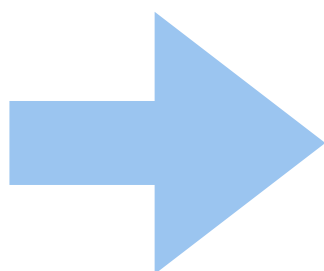
$$\text{Loss}(T, D) = \sum_{(f,v) \in D} (T(f) - v)^2$$

- Fit a model tree T to minimize the loss:

initial p	initial z	initial flip	Average post z
0.4	0	0	1.5
0.5	1	0	2
...			



Off-the-shelf
model tree
learning



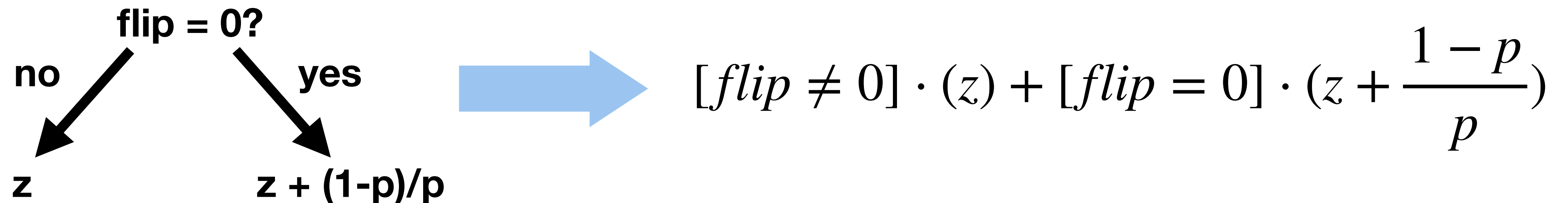
f : feature vector on s



v : $wpe(\text{while } flip = 0 \text{ do } \dots, z)(s)$

Learner: Extract Expectations

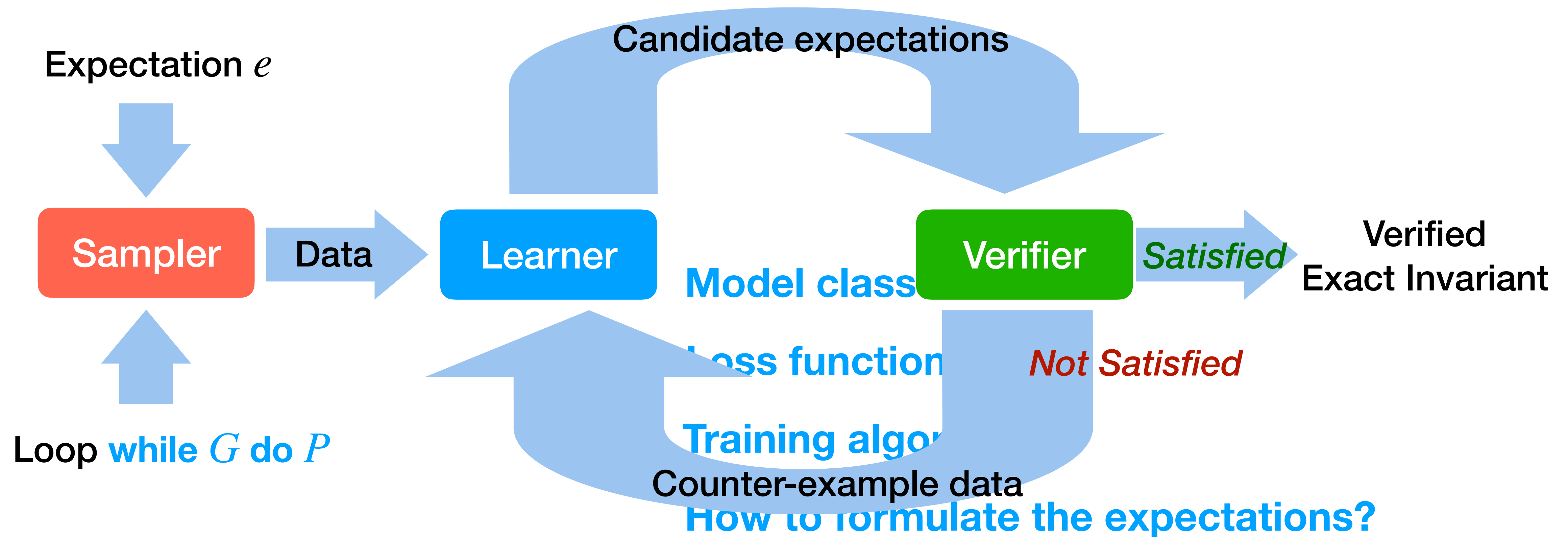
- A view change
 - Model Tree $T: \text{Features} \rightarrow \text{Numbers}$
 - Expectations: $\text{Program States} \rightarrow \text{Numbers}$
 - Let $M: \text{Program States} \rightarrow \text{Features}$, then $T \circ M$ is an expectation
- Represent as piece-wise mathematical expression



Estimate the map

Learn the map

Verify the expectation



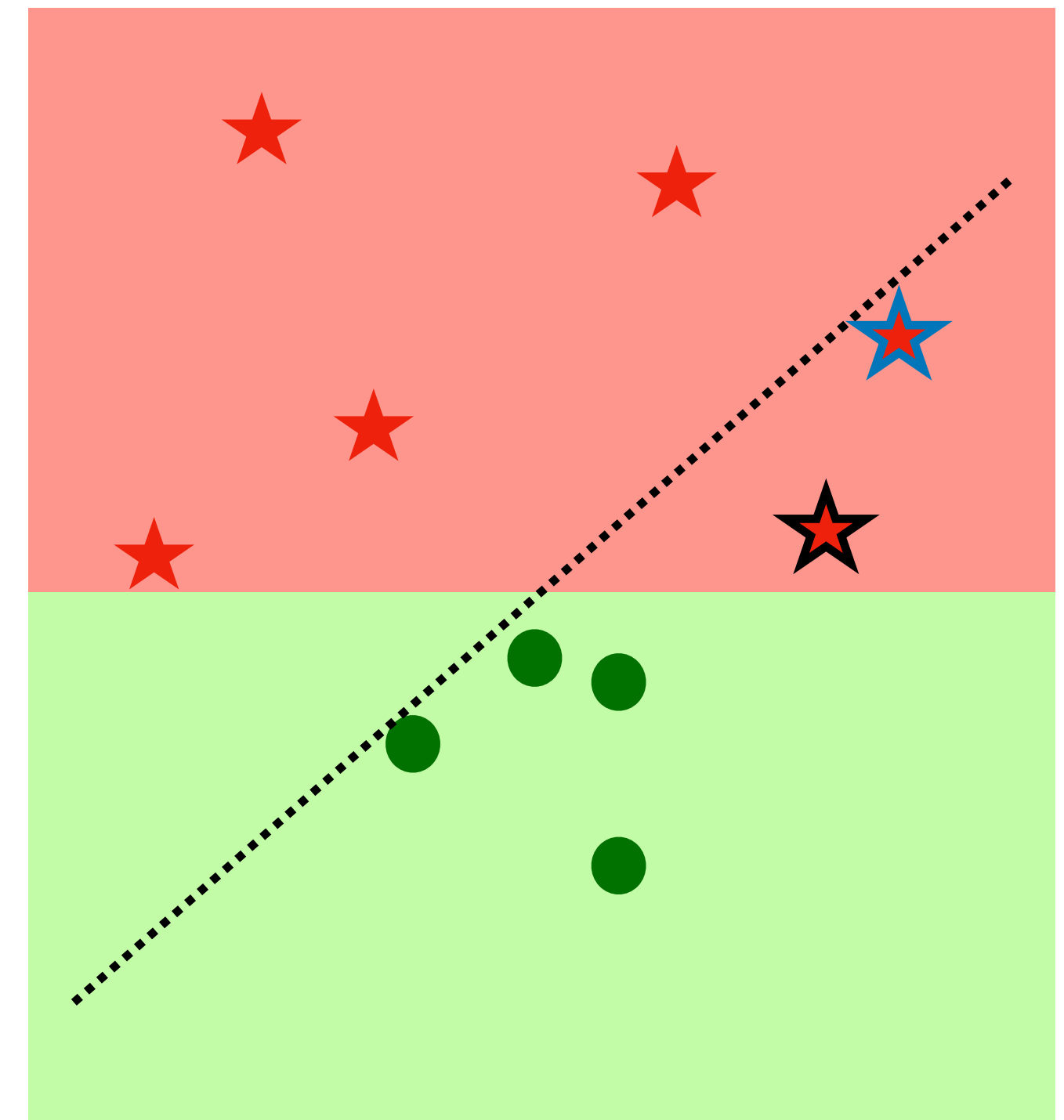
Verifier: Check Candidate Expectations

- **Recall:** In simple cases, $I = [G] \cdot wpe(P, I) + [\neg G] \cdot e$ iff $I = wpe(\text{while } G \text{ do } P, e)$.
- **Observation:** if P is loop less, then it's possible to calculate $wpe(P, I)$ syntactically
- Given candidate expectation I' , we use a solver to check if $I' = [G] \cdot wpe(P, I') + [\neg G] \cdot e$



Verifier: Find Counter-examples

- One counter-example may not change the learning process enough
- Solution:
 - Find multiple counter-examples
 - Find counter-example that maximize the violation



Implementation

- We implemented a prototype in Python, using Wolfram Alpha Engine for verifier
 - Sample 500 program states
 - Run 500 times from each program state
 - Timeout after 10 mins

Evaluations

- We evaluated on 18 benchmarks collected from prior work
 - Successfully generate invariants for 15 out of 18 benchmarks before timeout
 - Time cost: 3-299 sec
 - Dominated by sampling time

Limitations

- Fail when the ground truth exact invariant is too complicated
- Too many digits, e.g., $I = z + [n > 0] \cdot 2.625 \cdot n$
- Our learner oscillates between expectations like
 $[n > 0] \cdot 2.63 \cdot n - 0.02$ and $[n > 0] \cdot 2.62 \cdot n + 0.01$

Limitations

More data may help

- Fail when the ground truth exact invariant is too complicated
- Too many correlated terms, e.g.,
$$x \cdot y + [n > 0] \cdot (0.25 \cdot n^2 + 0.5 \cdot n \cdot x + 0.5 \cdot n \cdot y - 0.25 \cdot n)$$
- Our learner generates
$$\dots (0.25 \cdot n^2 + 0.5 \cdot n \cdot x + 0.5 \cdot n \cdot y - 0.27 \cdot n - 0.01 \cdot x + 0.12)$$

Limitations

Not only need more data but also more powerful verifier

- Fail when the ground truth exact invariant is too complicated
- The verifier gets stuck, e.g.,

$$[c = 0] \cdot [t = 0] \cdot \frac{p1}{p1 + p2 - p1 \cdot p2} +$$
$$[c = 0] \cdot [t = 0] \cdot \frac{(1 - p2) \cdot p1}{p1 + p2 - p1 \cdot p2} + [c = 0] \cdot (t)$$

Stepping Back ...

- Previously, we generate **exact invariant** by learning I that approximates $wpe(\text{while } G \text{ do } P, e)$ and then check if $I = [G] \cdot wpe(P, I) + [\neg G] \cdot e$.
- Question: Can we generate **subinvariants**, i.e., I such that $I \leq [G] \cdot wpe(P, I) + [\neg G] \cdot e$?
 - Sometimes the exact invariant is too complicated
 - I is a subinvariant implies $I \leq wpe(\text{while } G \text{ do } P, e)$, not the other way

New Problem: Learning subinvariants

- **Given:** a loop **while** G **do** P and two expectation pre and e .
- **Goal:** find expectation I such that $I \leq [G] \cdot wpe(P, I) + [\neg G] \cdot e$ and $pre \leq I$.

- **Equivalent conditions:**

$$\bigwedge_s I(s) \leq [G] \cdot wpe(P, I)(s) + [\neg G] \cdot e(s) \wedge \bigwedge_s pre(s) \leq I(s)$$

Casting into a Learning Problem

- The condition

- $\bigwedge_s I(s) \leq [G] \cdot wpe(P, I)(s) + [\neg G] \cdot e(s) \wedge \bigwedge_s preE(s) \leq I(s)$

- The ideal loss

- $Loss'(I) = \sum_s \max(0, I(s) - G(s) \cdot wpe(P, I)(s) - (1 - G(s)) \cdot e(s))$
 $+ \sum_s \max(0, preE(s) - I(s))$

In practice: we only sum over sampled states

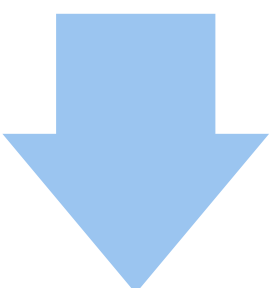
In practice: we need to estimate $\lambda I \cdot wpe(P, I)(s)$

How to estimate $\lambda I . wpe(P, I)(s)$



Data

Given I



$I(s_1)$

$I(s_2)$

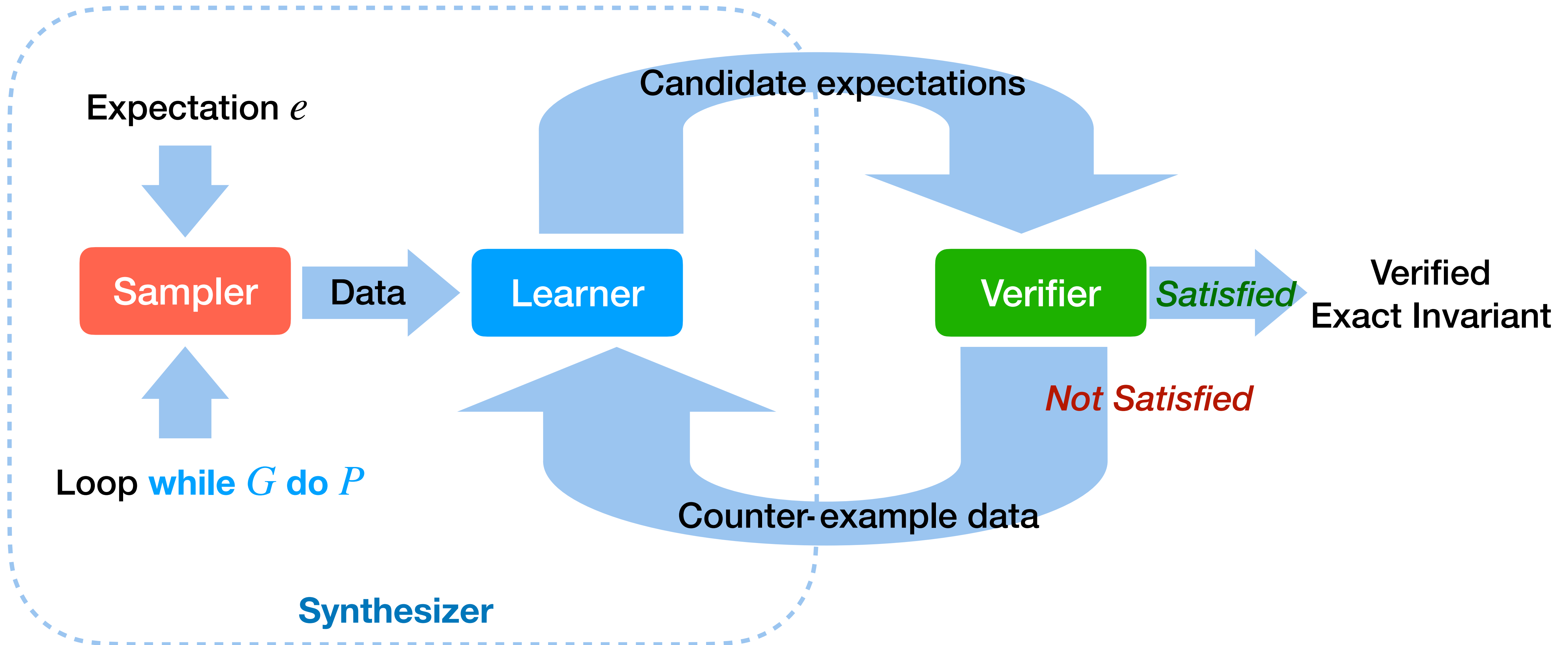
$I(s_3)$

...

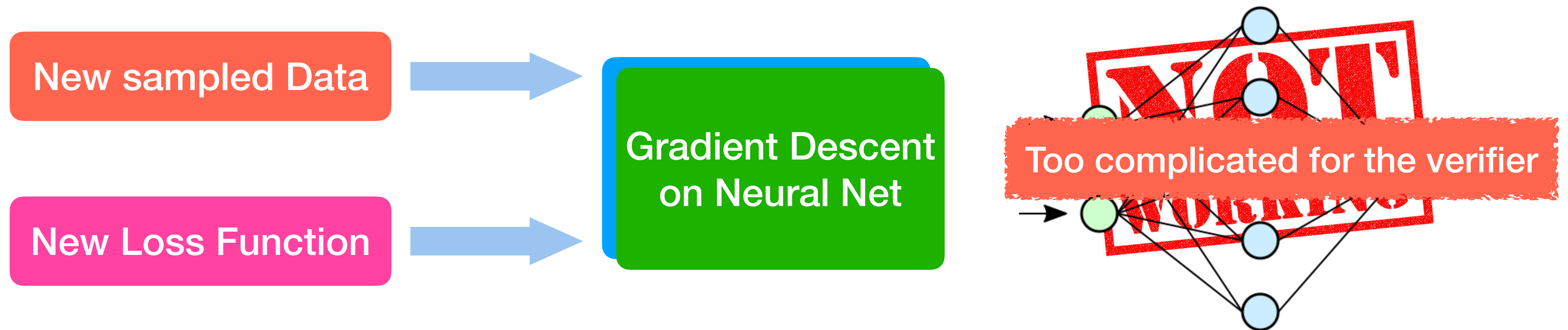


Their average estimates
 $wpe(P, I)(s)$

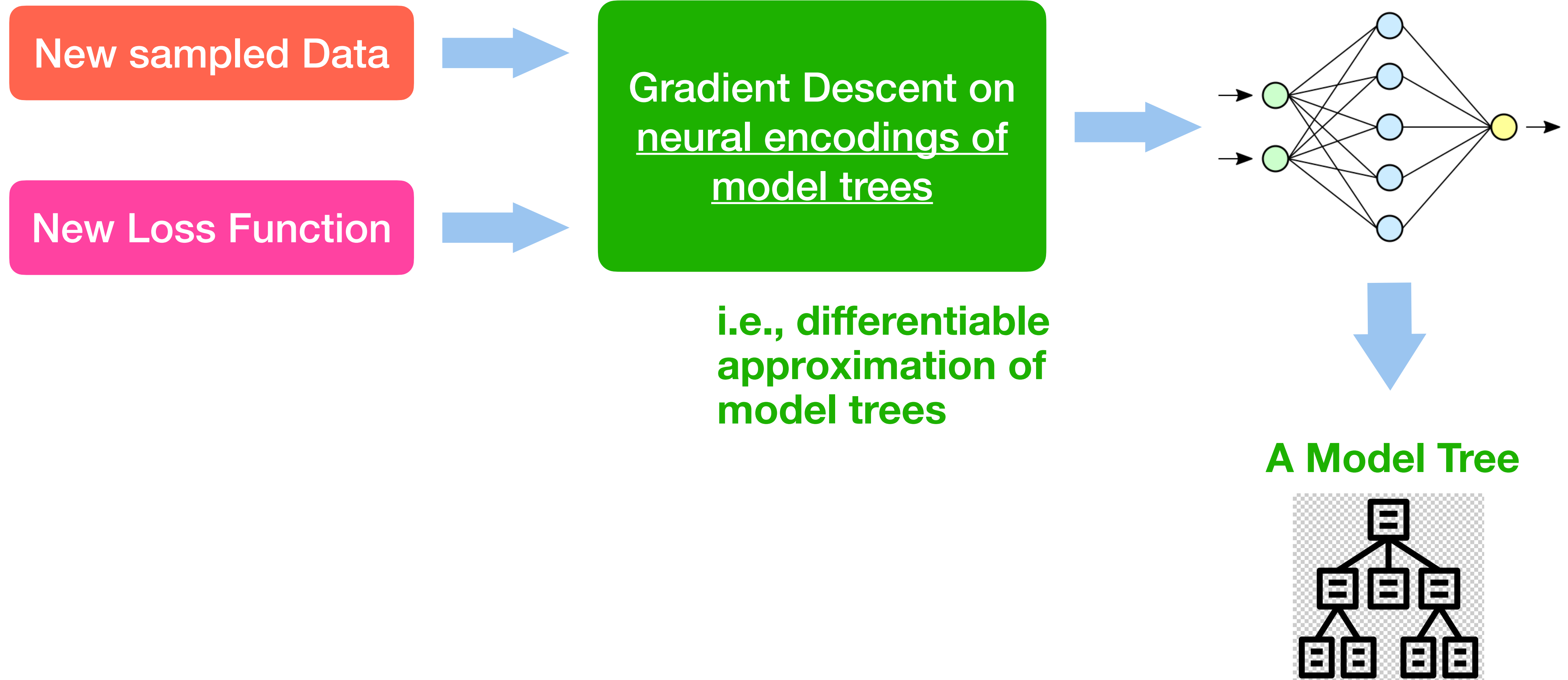
Same Method, Different Implementations



Challenge



Our Solution



Evaluations

- We constructed 32 benchmarks using the 18 programs we collected from prior work and different preexpectations
- Successfully generate subinvariants for 25 out of 32 benchmarks before timeout
- Time cost: 33-196 sec
 - Dominated by **learning** time

Limitations

- Noise in sample data
- Gradient descent seems to get stuck in local minima
 - It seems a hard learning program.

The Learning Problem

- Symbolic Regression:
 - Given: a dataset (X, y) where each point has inputs $X_i \in \mathbb{R}^n$ and response $y_i \in R$:
 - Goal: find a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ that best fits the data set, where f is a short closed-form mathematical expression.
- Our learning problem in exact invariant generation is almost the same.
- Our learning problem in subinvariant generation is a bit more general.

The State of Art of Symbolic Regression

Table 1: Recovery rate of several algorithms on the Nguyen benchmark problem set across 100 independent training runs. Results of our algorithm are obtained using PQT; slightly lower recovery rates were obtained using VPG and RSPG training (see Table 3 for comparisons).

Benchmark	Expression	Recovery rate (%)					
		Ours	DSR	PQT	VPG	GP	Eureqa
Nguyen-1	$x^3 + x^2 + x$	100	100	100	96	100	100
Nguyen-2	$x^4 + x^3 + x^2 + x$	100	100	99	47	97	100
Nguyen-3	$x^5 + x^4 + x^3 + x^2 + x$	100	100	86	4	100	95
Nguyen-4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$	100	100	93	1	100	70
Nguyen-5	$\sin(x^2) \cos(x) - 1$	100	72	73	5	45	73
Nguyen-6	$\sin(x) + \sin(x + x^2)$	100	100	98	100	91	100
Nguyen-7	$\log(x + 1) + \log(x^2 + 1)$	97	35	41	3	0	85
Nguyen-8	$\sqrt{x}$	100	96	21	5	5	0
Nguyen-9	$\sin(x) + \sin(y^2)$	100	100	100	100	100	100
Nguyen-10	$2 \sin(x) \cos(y)$	100	100	91	99	76	64
Nguyen-11	x^y	100	100	100	100	7	100
Nguyen-12	$x^4 - x^3 + \frac{1}{2}y^2 - y$	0	0	0	0	0	0
Average		91.4	83.6	75.2	46.7	60.1	73.9

From *Symbolic Regression via Neural-Guided Genetic Programming Population Seeding* [Neurips 2021]

Take Away

